EXPERIMENT 10: EXPLORE AND IMPLEMENT AUTOENCODERS FOR IMAGE DENOISING

1. DATASET SOURCE

The dataset used for this experiment is the **MNIST handwritten digits dataset**.

- Link: http://yann.lecun.com/exdb/mnist/

2. DATASET DESCRIPTION

The MNIST dataset is one of the most widely used benchmark datasets in machine learning and computer vision, particularly for image classification and representation learning tasks.

Key Characteristics:

- **Total Samples:** 70,000 images
  - Training set: 60,000
  - Test set: 10,000
- **Image Size:** 28 × 28 pixels
- **Color Format:** Grayscale (single channel)
- **Pixel Value Range:** 0 to 255
- **Number of Classes:** 10 (digits from 0 to 9)

Features:

Each image is represented as a 2D matrix of size 28×28, which can also be flattened into a 784-dimensional vector.

Target Variable:

Although MNIST is typically used for classification, in this experiment:

- **Input:** Noisy image
- **Target:** Clean image

Thus, the task is **image reconstruction**, not classification.

Dataset Transformation for This Experiment:

- Noise is artificially added (e.g., Gaussian noise)
- The model learns a mapping:

$$\text{Noisy Image} \rightarrow \text{Clean Image}$$

3. MATHEMATICAL FORMULATION OF THE ALGORITHMS

An **Autoencoder** is an unsupervised neural network that learns to reconstruct input data through a compressed representation.

Architecture Components:

1. **Encoder Function**

$$h = f(x) = \sigma(Wx + b)$$

2. **Latent Representation (Bottleneck)**
   Lower-dimensional encoding capturing essential features

3. **Decoder Function**

$$\hat{x} = g(h) = \sigma(W'h + b')$$

4. **Objective Function (Reconstruction Loss)**

Most commonly used loss:

$$L(x, \hat{x}) = \| x - \hat{x} \|^2$$

For binary images (like MNIST), Binary Cross Entropy is often used:

$$L = -\sum(x\log \hat{x} + (1 - x)\log (1 - \hat{x}))$$

Denoising Autoencoder Formulation

Instead of feeding clean input x, we use a corrupted version $\tilde{x}$:

$$h = f(\tilde{x})$$
$$\hat{x} = g(h)$$

Loss:

$$L(x, g(f(\tilde{x})))$$

The model learns:

$$g(f(\tilde{x})) \approx x$$

4. ALGORITHM LIMITATIONS

Although autoencoders are powerful, they have several limitations:

1. Dependence on Training Data

- Poor generalization if test noise differs from training noise
- Learns dataset-specific patterns rather than universal features

2. Overfitting Risk

- May memorize inputs instead of learning meaningful representations
- Especially when latent space is too large

3. Blurred Reconstructions

- MSE loss often leads to **smooth/blurred outputs**
- Fine details may be lost

4. Not Fully Generative

- Unlike VAEs or GANs, standard autoencoders cannot generate new samples effectively

5. Sensitivity to Noise Type

- Model trained on Gaussian noise may fail on salt-and-pepper noise
-

6. Computational Cost

- Deep architectures require significant training time and GPU resources

5. METHODOLOGY / WORKFLOW

Step 1: Data Loading

- Load MNIST dataset
- Normalize pixel values to [0,1]

Step 2: Noise Injection

- Add Gaussian noise:

$$\tilde{x} = x + \mathcal{N}(0, \sigma^2)$$

Step 3: Model Architecture Design

Typical architecture:

- Input layer: 784 neurons
- Encoder:
  - Dense layers (e.g., 128 → 64 → 32)
- Bottleneck:
  - Latent space (e.g., 16 dimensions)
- Decoder:
  - Reverse structure (32 → 64 → 128 → 784)

Step 4: Model Training

- Input: Noisy images
- Output: Clean images
- Loss: Binary Cross Entropy / MSE
- Optimizer: Adam

Step 5: Evaluation

- Compare original, noisy, and reconstructed images
- Compute reconstruction loss

6. PERFORMANCE ANALYSIS

Evaluation Metrics:

1. Mean Squared Error (MSE)

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

2. Peak Signal-to-Noise Ratio (PSNR)

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{\mathrm{MAX}^2}{\mathrm{MSE}}\right)$$

3. Structural Similarity Index (SSIM) (optional)

- Measures perceptual similarity

Observations:

- The model successfully removes a significant portion of noise
- Reconstructed digits are recognizable
- Some fine strokes may be blurred
- Performance improves with deeper architectures

Interpretation:

- Lower MSE → better reconstruction
- Higher PSNR → better image quality
- Visual inspection is equally important in denoising tasks

7. HYPERPARAMETER TUNING

Key Hyperparameters:

Hyperparameter	Effect
Latent dimension	Controls compression level
Learning rate	Affects convergence speed
Batch size	Impacts training stability
Number of layers	Affects representation power
Noise level	Determines difficulty of task

Tuning Strategy:

1. **Grid Search / Manual Tuning**
   - Try latent sizes: 8, 16, 32
   - Learning rates: 0.001, 0.0005

2. **Observations:**
   - Smaller latent space → higher compression but more loss
   - Larger latent space → better reconstruction but risk of overfitting
   - Moderate noise gives best learning balance

CONCLUSION

This experiment demonstrates how autoencoders can effectively learn meaningful representations for image denoising. By training on noisy MNIST images, the model successfully reconstructs clean images, showcasing the power of unsupervised learning in feature extraction and noise removal. However, limitations such as blurring and overfitting highlight the need for advanced variants like **Denoising Variational Autoencoders (VAE)** or **GAN-based models** for improved performance.